

Clase 294 — Robo y extracción de modelos

Parte: **15 — Seguridad de IA y machine learning** · Fuente: *Tramèr et al., "Stealing Machine Learning Models via Prediction APIs" (USENIX Security 2016)* 🕒 Duración estimada: **110 min** · Nivel: **Intermedio**

🎯 Objetivo

Aprender cómo un adversario con solo acceso a la API de predicción puede robar la funcionalidad de un modelo (model extraction), reconstruir datos de entrenamiento (model inversion) o determinar si un registro estuvo en el entrenamiento (membership inference). El alumno ejecutará una extracción sobre un modelo propio y aplicará defensas como rate limiting, ruido en la salida y marcas de agua.

⚠️ **Ética:** todas las extracciones se hacen contra un modelo propio. Consultar masivamente APIs de terceros para copiar sus modelos viola los términos de servicio y puede ser ilegal.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** las tres familias: extracción, inversión e inferencia de membresía.
- 2. **Ejecutar** un ataque de extracción entrenando un modelo sustituto a partir de consultas.
- 3. **Medir** la fidelidad y precisión del sustituto frente al modelo víctima.
- 4. **Implementar** un ataque de membership inference y cuantificar su AUC.
- 5. **Aplicar** defensas: limitación de consultas, salida con menos información, ruido, watermarking y privacidad diferencial.

📖 Temas

#	Tema	Por qué importa
1	Model extraction / stealing	Roba IP y facilita ataques de caja negra
2	Modelo sustituto y transferibilidad	Copia funcional para atacar offline
3	Model inversion	Reconstruye entradas sensibles (rostros, datos)
4	Membership inference	Revela si un dato estuvo en el entrenamiento (privacidad)
5	Superficie de la API: confiancias vs. etiquetas	Cuánta info filtra cada respuesta
6	Defensas de salida y de tasa	Reducir señal y frecuencia de consultas
7	Watermarking y privacidad diferencial	Detectar copias y limitar fugas

Definiciones y características

- **Model extraction:** entrenar un modelo sustituto que replica al víctima usando pares (consulta, respuesta). *Característica:* efectivo incluso con solo etiquetas, mejor con probabilidades.
- **Fidelidad vs. precisión del sustituto:** fidelidad = coincidir con el víctima (aun en sus errores); precisión = acertar la verdad. Un buen robo maximiza fidelidad.
- **Model inversion:** reconstruir entradas representativas de una clase a partir de las salidas del modelo. *Característica:* riesgo de privacidad en modelos de rostros o salud.
- **Membership inference:** decidir si un registro concreto formó parte del entrenamiento. *Característica:* explota el overfitting; se mide con AUC.
- **Rate limiting:** limitar consultas por cliente/tiempo; encarece la extracción.
- **Output perturbation:** devolver top-k, redondear o añadir ruido a las probabilidades para reducir la señal robable.
- **Privacidad diferencial:** entrenar con ruido calibrado (ϵ) para acotar cuánto influye un registro individual; mitiga membership inference.

Herramientas y preparación

```
pip install adversarial-robustness-toolbox torch torchvision scikit-learn numpy
```

- **ART** implementa `CopycatCNN` / `KnockoffNets` (extracción) y ataques de inferencia de membresía.
- Un modelo víctima propio, expuesto localmente como función `predict(x)`.
- Un dataset "de consulta" (puede ser distinto al de entrenamiento del víctima).

Laboratorio guiado

Sobre un **modelo víctima propio**.

1. **Prepara la víctima.** Entrena un clasificador y envuélvelo como una API local `predict()` que devuelve probabilidades.
2. **Presupuesto de consultas.** Fija un budget (p. ej. 10 000 consultas) para simular un atacante realista.
3. **Extrae con KnockoffNets.**

```
python from art.attacks.extraction import KnockoffNets attack =  
KnockoffNets(classifier=victim, batch_size_query=64, nb_epochs=10, nb_stolen=10000) stolen =  
attack.extract(x_query, thieved_classifier=surrogate)
```

4. **Mide fidelidad y precisión.** Compara las predicciones del sustituto con las del víctima (fidelidad) y con la verdad (precisión). Repite reduciendo la salida a solo etiquetas (top-1) y observa la caída.
5. **Usa el sustituto para atacar.** Genera adversariales de caja blanca sobre el sustituto y compruébalos contra el víctima (transferibilidad): así el robo habilita evasión.
6. **Membership inference.**

```
python from art.attacks.inference.membership_inference import MembershipInferenceBlackBox mi =  
MembershipInferenceBlackBox(victim) mi.fit(x_train, y_train, x_test, y_test)
```

Calcula el AUC: cuanto más overfitting, más alto.

7. **Defiende.** Aplica: (a) devolver solo top-1, (b) redondear probabilidades, (c) rate limiting simulado, (d) reentrenar con más regularización o privacidad diferencial. Reevalúa fidelidad del robo y AUC de membresía.

Ejercicios

1. Traza fidelidad del sustituto frente al número de consultas y encuentra el punto de saturación.
2. Compara robo con probabilidades completas vs. solo etiquetas.
3. Mide cómo el overfitting del víctima afecta al AUC de membership inference.
4. Implementa output perturbation y cuantifica cuánto degrada el robo y cuánto la utilidad legítima.
5. Diseña una política de rate limiting y estima el coste/tiempo que impone al atacante.
6. Explica cómo un watermark permitiría probar en un juicio que un modelo fue robado.

Reto verificable

Entrega un **estudio de extracción con defensa**: script que roba tu modelo alcanzando fidelidad $\geq 90\%$ con probabilidades completas, y demuestra que al aplicar una defensa (top-1 + ruido + rate limiting) la fidelidad cae de forma significativa manteniendo la utilidad legítima aceptable.

Criterio de aceptación: el informe presenta una tabla con fidelidad del sustituto antes y después de la defensa, el AUC de membership inference antes y después, y una frase justificando el compromiso utilidad/seguridad elegido. Todo reproducible.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El sustituto no converge	Pocas consultas o dataset de consulta muy distinto. Aumenta el budget o acerca la distribución.
Membership inference con $AUC \approx 0.5$	El víctima no sobreajusta (bien) o el ataque está mal ajustado. Verifica el split train/test.
La defensa mata la utilidad	Ruido excesivo. Calibra ϵ o el redondeo hasta equilibrar.
Rate limiting inefectivo	El atacante rota IPs/cuentas. Combina con detección de patrones de consulta anómalos.
Creer que top-1 basta	Aun con solo etiquetas se puede robar; añade otras capas (tasa, monitorización).

Preguntas frecuentes

 **¿Puedo robar un modelo solo con etiquetas, sin probabilidades?** Sí, aunque necesitas más consultas. Las probabilidades aceleran mucho el robo porque filtran más información por consulta.

 **¿La extracción es solo un problema de propiedad intelectual?** No. El sustituto habilita ataques de caja negra: generas adversariales sobre tu copia y los transfieres al modelo real.

? **¿La privacidad diferencial resuelve el membership inference?** Lo mitiga acotando la influencia de cada registro, pero con coste de utilidad. Elegir ϵ es un compromiso privacidad/precisión que debe justificarse.

? **¿Qué es el watermarking de modelos?** Incrustar un comportamiento secreto (respuestas concretas a entradas clave) que solo el dueño conoce, para demostrar la autoría de un modelo sospechoso de ser copia.

Referencias

- Tramèr et al., "Stealing Machine Learning Models via Prediction APIs", USENIX 2016 — <https://arxiv.org/abs/1609.02943>
- Shokri et al., "Membership Inference Attacks Against ML Models", IEEE S&P 2017 — <https://arxiv.org/abs/1610.05820>
- Fredrikson et al., "Model Inversion Attacks", CCS 2015.
- Adversarial Robustness Toolbox — <https://github.com/Trusted-AI/adversarial-robustness-toolbox>
- MITRE ATLAS, "Exfiltration via ML Inference API" — <https://atlas.mitre.org/>

Siguiente clase

Clase 295 - OWASP Top 10 para aplicaciones con LLM